

J2 – DTOs : avant / après

Branche `day2/dtos-mapstruct`. On remplace les classes par des records, et le mapping manuel par MapStruct. Bean Validation reste sur la requête.

Fichier	Changement
dto/ProductDto.java	classe → record
dto/CreateProductRequest.java	classe → record, mêmes annotations
dto/ProductPageDto.java	classe → record
mapper/ProductMapper.java	fichier nouveau
service/ProductService.java	le service appelle le mapper

Un record n'a pas de setters. On lit un champ avec `request.categoryId()`, plus `request.getCategoryId()`. Le JSON ne change pas : Jackson écrit toujours `"categoryId"`.

1. ProductDto

`src/main/java/tn/clevery/api/dto/ProductDto.java`

DTO de sortie. Avant : 7 champs, un constructeur, 14 getters/setters. Après : la liste des champs suffit. Le record est immuable.

AVANT

```
public class ProductDto {

    private Long id;
    private String name;
    private BigDecimal price;
    private String description;
    private Long categoryId;
    private String categoryName;
    private Instant createdAt;

    public ProductDto() {}

    public ProductDto(Long id, String name, BigDecimal price, String description,
                      Long categoryId, String categoryName, Instant createdAt) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.description = description;
        this.categoryId = categoryId;
        this.categoryName = categoryName;
        this.createdAt = createdAt;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public BigDecimal getPrice() { return price; }
    public void setPrice(BigDecimal price) { this.price = price; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public Long getCategoryId() { return categoryId; }
    public void setCategoryId(Long categoryId) { this.categoryId = categoryId; }
    public String getCategoryName() { return categoryName; }
    public void setCategoryName(String categoryName) { this.categoryName = categoryName; }
    public Instant getCreatedAt() { return createdAt; }
    public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }
}
```

APRÈS

```

public record ProductDto(
    Long id,
    String name,
    BigDecimal price,
    String description,
    Long categoryId,
    String categoryName,
    Instant createdAt
) {}

```

2. CreateProductRequest

src/main/java/tn/clevory/api/dto/CreateProductRequest.java

DTO d'entrée. Les annotations ne bougent pas : elles passent sur les composants du record. `@Valid` dans le contrôleur continue de les déclencher.

AVANT

```

public class CreateProductRequest {

    @NotBlank(message = "Le nom est obligatoire")
    @Size(min = 2, max = 120, message = "Le nom doit contenir entre 2 et 120 caractères")
    private String name;

    @NotNull(message = "Le prix est obligatoire")
    @Positive(message = "Le prix doit être strictement positif")
    @Digits(integer = 10, fraction = 2)
    private BigDecimal price;

    @NotNull(message = "La catégorie est obligatoire")
    private Long categoryId;

    @Size(max = 2000, message = "La description ne peut dépasser 2000 caractères")
    private String description;

    public CreateProductRequest() {}

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public BigDecimal getPrice() { return price; }
    public void setPrice(BigDecimal price) { this.price = price; }
    public Long getCategoryId() { return categoryId; }
    public void setCategoryId(Long categoryId) { this.categoryId = categoryId; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
}

```

APRÈS

```

public record CreateProductRequest(
    @NotBlank(message = "Le nom est obligatoire")
    @Size(min = 2, max = 120, message = "Le nom doit contenir entre 2 et 120 caractères")
    String name,

    @NotNull(message = "Le prix est obligatoire")
    @Positive(message = "Le prix doit être strictement positif")
    @Digits(integer = 10, fraction = 2)
    BigDecimal price,

    @NotNull(message = "La catégorie est obligatoire")
    Long categoryId,

    @Size(max = 2000, message = "La description ne peut dépasser 2000 caractères")
    String description
) {}

```

3. ProductPageDto

src/main/java/tn/clevory/api/dto/ProductPageDto.java

L'enveloppe de pagination. Même cinq champs. On supprime le constructeur vide et les setters : une page ne se modifie pas après création.

AVANT

```
public class ProductPageDto {

    private List<ProductDto> content;
    private int page;
    private int size;
    private long totalElements;
    private int totalPages;

    public ProductPageDto() {}

    public ProductPageDto(List<ProductDto> content, int page, int size,
                           long totalElements, int totalPages) {
        this.content = content;
        this.page = page;
        this.size = size;
        this.totalElements = totalElements;
        this.totalPages = totalPages;
    }

    public List<ProductDto> getContent() { return content; }
    public void setContent(List<ProductDto> content) { this.content = content; }
    public int getPage() { return page; }
    public void setPage(int page) { this.page = page; }
    public int getSize() { return size; }
    public void setSize(int size) { this.size = size; }
    public long getTotalElements() { return totalElements; }
    public void setTotalElements(long totalElements) { this.totalElements = totalElements; }
    public int getTotalPages() { return totalPages; }
    public void setTotalPages(int totalPages) { this.totalPages = totalPages; }
}
```

APRÈS

```
public record ProductPageDto(
    List<ProductDto> content,
    int page,
    int size,
    long totalElements,
    int totalPages
) {}
```

4. ProductMapper – fichier nouveau

src/main/java/tn/clevory/api/mapper/ProductMapper.java

Avant, ce fichier n'existait pas. Le mapping était une méthode privée `toDto` dans le service. MapStruct génère l'implémentation à la compilation (`@Mapper(componentModel = "spring")`), et Spring l'injecte.

AVANT

Fichier absent.
Le service copiait les champs à la main.

APRÈS

```

@Mapper(componentModel = "spring")
public interface ProductMapper {

    ProductDto toDto(Product entity);

    List<ProductDto> toDtoList(List<Product> entities);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "categoryName", ignore = true)
    @Mapping(target = "createdAt", ignore = true)
    Product toEntity(CreateProductRequest request);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "categoryName", ignore = true)
    @Mapping(target = "createdAt", ignore = true)
    void updateEntityFromRequest(CreateProductRequest request, @MappingTarget Product entity);
}

```

`ignore = true` sur `id`, `categoryName` et `createdAt` : ces champs ne viennent pas du body. `categoryName` est encore rempli par `CategoryRepository` dans le service.

5. ProductService

`src/main/java/tn/clevory/api/service/ProductService.java`

Les filtres et `delete` ne changent pas. On retire `toDto`, on injecte le mapper, et `getName()` devient `name()`.

AVANT – CREATE

```

Product entity = new Product(
    request.getName(),
    request.getPrice(),
    request.getCategoryId(),
    request.getDescription()
);
categoryRepository.findById(request.getCategoryId())
    .ifPresent(c -> entity.setCategoryName(c.getName()));
return toDto(repository.save(entity));

```

APRÈS – CREATE

```

Product entity = mapper.toEntity(request);
categoryRepository.findById(request.categoryId())
    .ifPresent(c -> entity.setCategoryName(c.getName()));
return mapper.toDto(repository.save(entity));

```

AVANT – UPDATE

```

entity.setName(request.getName());
entity.setPrice(request.getPrice());
entity.setCategoryId(request.getCategoryId());
entity.setDescription(request.getDescription());
categoryRepository.findById(request.getCategoryId())
    .ifPresent(c -> entity.setCategoryName(c.getName()));
return toDto(repository.save(entity));

```

APRÈS – UPDATE

```

mapper.updateEntityFromRequest(request, entity);
categoryRepository.findById(request.categoryId())
    .ifPresent(c -> entity.setCategoryName(c.getName()));
return mapper.toDto(repository.save(entity));

```

AVANT – MÉTHODE SUPPRIMÉE

```
private ProductDto toDto(Product p) {  
    return new ProductDto(  
        p.getId(), p.getName(), p.getPrice(), p.getDescription(),  
        p.getCategoryId(), p.getCategoryName(), p.getCreatedAt());  
}
```

APRÈS – LIST ET GETBYID

```
page.map(mapper::toDto)  
repository.findById(id).map(mapper::toDto)
```